ISA5810 DM2026 Lab 1 — Master Notebook Questions

Fill in this file as you work through DM2026-Lab1-Master.ipynb. Submit this completed file converted to PDF together with the rest of your Lab 1 work.

Please do not change the style of this document.

Question numbers 1–10 follow the same order as the notebook (top to bottom).

Student Information

Please complete the fields below. For **English Name** and **Chinese Name**, you may fill in **only one** if you do not have the other.

- **English Name: Pee Ge Zheng**
- **Chinese Name: 彭祉錚**
- **Email: gzgezheng@gmail.com**
- **Student ID: 115065424**
- **University: NTHU**

Instructions

These questions go with the Master Jupyter notebook (DM2026-Lab1-Master.ipynb). Each question is tied to a specific notebook section. Open the notebook, find **Question N (take home)**, and write your answer here.

We **strongly** suggest running the notebook to learn the lab, but the TAs will **not** audit whether you executed every cell to answer the questions in this document.

How to answer each question

1. Find the **notebook section** listed under the question (use the Table of Contents if needed).
2. Read the task; change parameters when asked (e.g. stop_words, thresholds, k, minSup).
3. Write a clear answer in the **Answer** space below each question.
4. Leave the question text unchanged; write only in the answer and disclosure fields.

Sources, honesty, and AI use

Acceptable vs. Unacceptable AI Usage: Generative AI should serve as an interactive tutor to guide your understanding, not as an automated solver completing the assignment for you.

- **Permitted (AI as Guide & Editor):**
  - Asking AI to explain unfamiliar theoretical concepts, discuss algorithmic trade-offs, or troubleshoot runtime errors in the notebook.
  - Polishing the grammar, phrasing, and readability of your own original, self-written drafts.

- **Prohibited (AI as Solver):**
  - Copying assignment questions directly into an AI tool to generate answers.
  - Using AI to fabricate experimental results or observations without executing the code in DM2026-Lab1-Master.ipynb.
  - Submitting machine-generated prose wholesale.

- **External material:** If you use websites, books, papers, or documentation beyond the notebook, cite them with **clickable links** (or DOI/ISBN when no URL exists).

- **When AI was used:** Your answer is acceptable if you (1) **disclose** AI use and (2) include **at least one external link** that validates the information in your answer (official docs, textbook, paper, or reputable tutorial — not the AI chat alone as the only source).

- **When AI was not used:** You may write "No" under **AI used** or leave it blank only if you truly did not use AI for that question. References are still encouraged if you looked anything up.

Under each question, complete:

- **AI used for this answer:** Yes / No
- **If yes, how:** (e.g. brainstorming, grammar, concept explanation)
- **External references (links):** *(required when AI = Yes; encouraged otherwise)*

Write in English. Keep answers concise and specific.

Questions

Question 1: Data Filtering & Domain Relevance

Notebook: Section 3.2 — Question 1 (take home)

Task: Filter the dataframe **X** for the category **sci.med** and check its length.

Questions:

1. How many documents are in this specific category?
2. From a data mining perspective, why is isolating specific categories a crucial first step before applying algorithms like frequent pattern mining?

Answer:

1. 594
2. If we isolate the sci.med documents first, the word sets related to medicine will have a much higher support, because they are counted against 594 documents instead of all 2,257 documents. But when all the categories are mixed together, the support of these medical word sets drops below the minimum support threshold, so they are filtered out. As a result, the patterns we find will mostly be general words that all the categories use, instead of patterns that really represent the medical topic.

AI used for this answer: Yes

If yes, how: Concept explanation (support and minimum support in frequent pattern mining) and grammar polishing of my own draft.

External references (links): Han, Kamber & Pei, Data Mining: Concepts and Techniques (3rd ed.), Ch. 6, ISBN 978-0123814791

Question 2: Handling Missing Data

Notebook: Section 4.1 — Question 2 (take home) (after the **dropna()** demonstration)

Task: In Section 4.1, a dummy record with a missing value is injected and then removed using the **dropna()** function.

Question: If 40% of documents were missing the **category_name** label, why would simply dropping these rows (with **dropna()**) be a flawed data mining strategy? Name one objective alternative so the text data can still be utilized.

Answer: Dropping these rows would be a flawed strategy because we would lose 40% of the data, even though their text is still complete and useful. It could also bias the dataset, since the missing labels may not be evenly distributed across the categories. Instead, we can use the 60% of documents that have labels to train a machine learning classifier on the text, and then use it to predict the missing category_name for the remaining 40%.

AI used for this answer: Yes

If yes, how: Concept clarification (using a classifier to predict missing labels, sampling bias), and grammar polishing of my own draft

External references (links):
https://pandas.pydata.org/docs/user_guide/missing_data.html

Question 3: Data Sampling Risks

Notebook: Section 5.1 — Question 3 (take home)

Task: In Section 5.1, the code uses **X_sample = X.sample(n=1000)** to create a subset. Compare the bar chart of the original category distribution to the sampled distribution.

Questions:

1. Do the exact category proportions perfectly match the original?
2. What is the specific risk of simple random sampling on a highly imbalanced dataset?

Answer:

1. No, the proportions do not perfectly match. For example, comp.graphics increased from 25.9% to 27.8% (+1.9 percentage points), while soc.religion.christian decreased from 26.5% to 24.8% (−1.7 points). This happens because random sampling naturally introduces small variations (sampling error), and since the proportions must add up to 100%, when one category is over-sampled, another must be under-sampled.
2. With simple random sampling on a highly imbalanced dataset, a minority class may be under-represented or even completely missing from the sample by chance, which biases any analysis or model toward the majority classes. Stratified sampling avoids this by sampling each category separately according to its original proportion.

AI used for this answer: Yes / No

If yes, how: help phrasing my own ideas in English

External references (links):

- pandas DataFrame.sample -

  https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.sample.html

- https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

---

Question 4: Text Vectorization and Matrix Sparsity

Notebook: Section 5.3 — Question 4 (take home)

Task: In Section 5.3, the **CountVectorizer** converts text into a document-term matrix using **stop_words='english'**. Change this parameter to **stop_words=None** and re-run the cell to check the shape of **X_counts**.

Questions:

1. Does the total number of features (columns) increase or decrease?
2. Since stopwords are highly common words that appear in almost all documents, what effect does adding them back have on the overall sparsity (the percentage of zeros) of the matrix?

Answer:

1. The number of features increases, from 35,482 (stop_words='english') to 35,788 (stop_words=None), which adds 306 stopword features.

2. The sparsity decreases slightly, from 99.6772% to 99.5470%. Stopwords like "the" and "is" appear in almost every document, so their columns are mostly non-zero. Adding them back fills in many cells, which lowers the overall percentage of zeros, but these extra features carry little useful information for distinguishing between categories.

AI used for this answer: No

If yes, how:

External references (links):

- scikit-learn, sklearn.feature_extraction.text.CountVectorizer: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html
- scikit-learn User Guide, Text feature extraction: https://scikit-learn.org/stable/modules/feature_extraction.html#text-feature-extraction

---

Question 5: Feature Selection — Variance Thresholding

Notebook: Section 5.3.1 — Question 5 (take home)

Task: In the variance filtering section, re-run **variance_filter** using threshold values **threshold = 0.0, threshold = 0.01, and threshold = 0.05**. Record the number of features marked as **keep == True**.

Questions:

1. What happens to the total size of the vocabulary as the variance threshold increases from 0.0 to 0.05?
2. In terms of document frequency, what kinds of words have variance close to zero and are eliminated first?
3. Why is eliminating near-zero variance features considered a safe way to reduce dimensionality without harming model accuracy?

Answer:

Threshold	Number of terms kept (keep == True)
0.0	35482
0.01	5089
0.05	1491

1. The vocabulary size drops sharply as the threshold increases (see table). The decrease is not linear: most terms are removed with only a small increase from 0.0 to 0.01, because the majority of words are very rare and have variance close to zero. At 0.0, no terms are removed since every term appears in at least one document, so its variance is above zero.
2. Words with very low document frequency are removed first. At threshold = 0.01, the median document frequency of the removed words is 1, and the maximum is only 22 out of 2,257 documents. Examples include names, abbreviations, and rare or misspelled words such as “zubkoff”, “zsoft”, and “zues”.
3. These near-zero variance words appear in so few documents that they carry almost no information for distinguishing between categories, and they are unlikely to appear in new documents. A model cannot learn a useful pattern from them, so removing them barely affects accuracy, while greatly reducing the number of features and making computation faster.

AI used for this answer: Yes

If yes, how: Concept explanation (variance and threshold)

External references (links): (Removing features with low variance):https://scikit-learn.org/stable/modules/feature_selection.html#removing-features-with-low-variance

---

Question 6: Feature Correlation — Linear (Pearson) vs. Monotonic (Spearman)

Notebook: Section 5.3.2 — Question 6 (take home)

Task: Run the correlation filter against the binary indicator for the **comp.graphics** class. Inspect the top 10 features sorted by **pearson_r** and compare them to the top 10 features sorted by **spearman_r**.

Questions:

1. Are the top-correlated words generic stop words or domain-specific terminology for graphics?
2. What is the mathematical difference in what **Pearson** correlation measures versus what **Spearman** rank correlation measures?
3. In what specific scenario would a word have a high **Spearman** rank correlation with a topic, but a significantly lower **Pearson** correlation?

Answer:

1. The Pearson top 10 includes several generic forum or email-header words such as “thanks”, “hi”, “host”, and “nntp” (not classic stop words, which were already removed), mixed with a few graphics terms like “vga” and “video”. The Spearman top 10 is almost entirely domain-specific graphics terminology, such as “graphics”, “image”, “3d”, and “format”.
2. Pearson measures a linear relationship using the raw counts, so it is sensitive to the size of the values and to outliers. Spearman converts the counts into ranks and measures a monotonic relationship, only checking whether higher counts tend to go with the graphics class.
3. When a word appears consistently in the target category but a few documents contain extreme counts. For example, “graphics” appears mostly once or twice per document but 105 times in one, giving a low Pearson r (0.178) but the highest Spearman r (0.521), because ranking removes the effect of the outlier.

AI used for this answer: Yes

If yes, how: Concept explanation (Pearson vs. Spearman, effect of outliers)

External references (links):

- SciPypearsonr:https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.pearsonr.html
- SciPyspearmanr:https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.spearmanr.html

---

Question 7: Frequent Pattern Mining — Algorithm & Parameter Dynamics

Notebook: Section 5.4.2 — Question 7 (take home)

Task: In the Frequent Pattern Mining section, run **Top-K FAE** with **k = 100 and k = 500**. Then, run **MaxFPGrowth** using minimum support thresholds **minSup=9 and minSup= 3**.

Questions:

1. How does decreasing **minSup** (from 9 to 3) in **MaxFPGrowth** affect the execution runtime and the total number of discovered patterns?
2. How does the parameter k in **Top-K FAE** differ fundamentally from **minSup** in **MaxFPGrowth** when deciding which patterns to keep?
3. Why are "maximal" frequent patterns structurally more concise than standard frequent patterns?

Answer (1) — Runtime and pattern count when lowering minSup:

Both increase sharply: the runtime rose from 0.11 s to 1.85 s, and the number of patterns from 710 to 9,547. A lower threshold prunes fewer itemsets, so longer, rarer combinations also qualify (average length grew from 1.13 to 3.11 words).

Answer (2) — Top-K k vs MaxFPGrowth minSup:

minSup fixes the support threshold, so the number of patterns varies. k fixes the number of patterns, so the threshold adjusts automatically (lowest support kept: 21 for k = 100, 16 for k = 500).

Answer (3) — Maximal vs standard frequent patterns:

A maximal pattern has no frequent superset. Since every subset of a frequent itemset is also frequent, keeping only the largest one e.g {A, B, C} represents all its subsets without listing them separately, removing redundancy.

AI used for this answer: Yes

If yes, how: Concept explanation (minSup, Top-K, maximal patterns)

External references (links): GitHub:https://github.com/UdayLab/PAMI

Question 8: Dimensionality Reduction — Motivation

Notebook: Section 5.5 — Question 8 (take home) (before PCA/t-SNE code)

Task: After text vectorization, the **X_counts** matrix creates tens of thousands of distinct features (words).

Questions:

1. If we want to visually map how these documents group by category on a scatter plot, why is it mathematically and visually impossible to plot the raw **X_counts** matrix directly?
2. What specific category of data mining algorithms (referenced in the helper files) must be applied first?

Answer:

1. A scatter plot only has 2 (or at most 3) axes, but each document in X_counts has 35,482 dimensions (one per word). These cannot be drawn on a 2D plane, and humans cannot visually interpret more than three dimensions.

2. Dimensionality reduction algorithms, such as PCA, t-SNE, and UMAP, must be applied first to project the data into 2 or 3 dimensions while keeping its structure.

AI used for this answer: Yes

If yes, how: Concept explanation (why high-dimensional data cannot be plotted directly) and help phrasing my answers in English

External references (links): scikit-learn Decomposition / PCA

https://scikit-learn.org/stable/modules/decomposition.html#pca

---

Question 9: Dimensionality Reduction — Linear (PCA) vs. Non-linear Manifolds (t-SNE/UMAP)

Notebook: Section 5.5 — Question 9 (take home) (after 2D/3D plots)

Task: In the 2D/3D Dimensionality Reduction section, compare the scatter plot projection generated by **PCA** against that generated by **t-SNE** or **UMAP**.

Questions:

1. Which method **(PCA or t-SNE/UMAP**) shows distinct, separated cluster islands for the 4 document categories, and which shows heavy overlap around the center?
2. Why does **PCA** struggle to separate high-dimensional text categories compared to **t-SNE** or **UMAP**?
3. Why must data analysts be careful not to over-interpret the physical distance between separate clusters in a **t-SNE** plot?

Answer:

1. t-SNE and UMAP show more distinct cluster regions for the categories (e.g., alt.atheism and soc.religion.christian form separate areas, and sci.med forms small dense islands), although some overlap remains in the center. PCA shows heavy overlap: almost all documents are compressed into one spot near the origin.

2. PCA is a linear method that projects the data onto the directions of maximum variance. In this sparse text data, that variance is dominated by a few extreme comp.graphics documents, which stretch the axes, so the other documents get squeezed together. PCA also cannot capture the non-linear structure of text, while t-SNE and UMAP focus on preserving local neighborhoods, placing similar documents close together.

3. t-SNE only preserves local similarity: points that are close together are similar, but the distances between separate clusters, and their relative positions, are not meaningful. The layout can also change between runs e.g. appear mirrored, so we should not conclude that two clusters are more related just because they appear closer.

AI used for this answer: Yes

If yes, how: Concept explanation (PCA vs. t-SNE/UMAP, interpreting t-SNE distances)

External references (links):

- scikit-learn, sklearn.manifold.TSNE: https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html
- Wattenberg, Viégas & Johnson, "How to Use t-SNE Effectively", Distill (2016): https://distill.pub/2016/misread-tsne/
- UMAP documentation: https://umap-learn.readthedocs.io/en/latest/

Question 10: One-Hot Binarization

Notebook: Section 5.6 — Question 10 (take home)

Task: In Section 3.1, examine **X['category']** (integers 0, 1, 2, 3). Then look at the output of **LabelBinarizer** in Section 5.6, where each category is converted into a binary vector (e.g., [1, 0, 0, 0]).

Questions:

1. How many binary columns are created to represent the 4 categories?
2. If an algorithm (like linear regression or KNN) is given the raw integer column **(0, 1, 2, 3)** as input, what false mathematical assumption does it make about the categories?
3. How does one-hot binarization eliminate this false assumption?

Answer:

1. 4 binary columns are created, one for each category (alt.atheism → [1, 0, 0, 0])
2. The algorithm wrongly assumes that the categories have a numerical order and meaningful distances, for example that soc.religion.christian (3) is "greater" than comp.graphics (1), or that alt.atheism (0) is more similar to comp.graphics (1) than to soc.religion.christian (3). These numbers are just arbitrary labels, so these assumptions are false.
3. One-hot binarization represents each category as a separate binary column, so there is no order between them, and every pair of different categories is the same distance apart. This treats all categories as equal and independent.

AI used for this answer: Yes

If yes, how: Concept explanation (integer vs. one-hot encoding) and help phrasing my answers in English

External references (links):

- scikit-learn, sklearn.preprocessing.LabelBinarizer: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelBinarizer.html
- scikit-learn User Guide, Encoding categorical features: https://scikit-learn.org/stable/modules/preprocessing.html#encoding-categorical-features

---